

Figure 6: Git branches

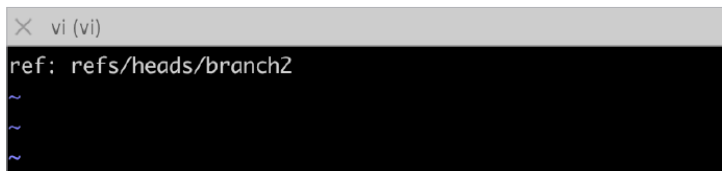


Figure 7: Editing the HEAD file

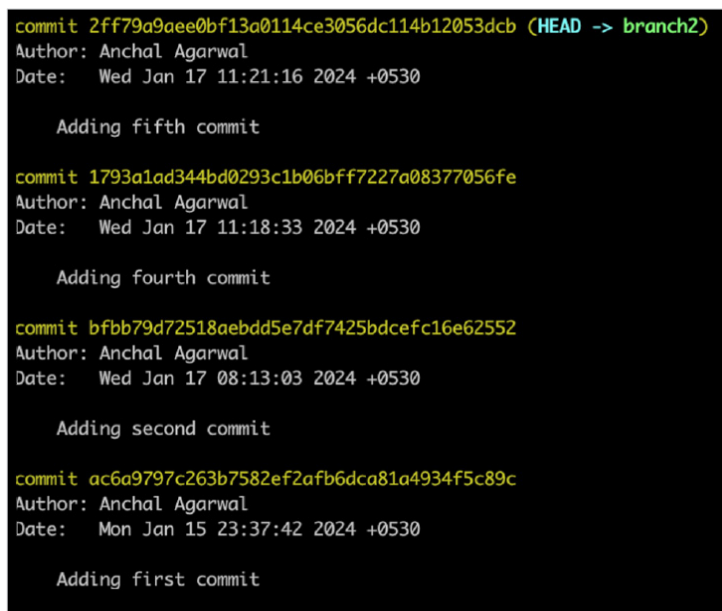


Figure 8: Output of *git log*

Git branches

While working with Git, you may have also come across branches. We create a branch wherever we want to work on different tasks in the same repository. A branch in Git is just a file pointing to a commit object ID. Yes, that's it. It stores nothing else.

The location where all branches are present is the `.git/refs/` folder. Creating a new branch is equal to adding a new file in this folder with a commit ID.

If you are following this tutorial from the beginning, then you should have only one branch as *main*. Also, if we check the *heads* folder, it only contains the *main* file right now.

Now add a new branch by creating a new file in this folder called *branch2*. Since I wanted this branch to point to the second commit ID, I have added a second commit ID in this file.

```

→ heads git:(main) echo
"bfb79d72518aebdd5e7df7425bdcefc16e62552" > branch2
→ heads git: (main) ls
branch2 main
-> heads git: (main) cat branch2
bfb79d72518aebdd5e7df7425bdcefc16e62552
→ heads git:(main)
  
```

If we check the *git log* history, a new branch as *branch2* has been created and points to the second commit.

If we enter *git branch*, we will be able to see two branches now.

If you see the *git log*, you will see a *HEAD* pointer to a branch. This *HEAD* pointer records the current branch of the repository. *HEAD* is a file present in the `.git` folder of a repository. In our case it is pointing to the *main* branch.

```

→ .git git: (main) cat HEAD
ref: refs/heads/main
→ .git git: (main)
  
```

The *HEAD* pointer changes whenever we do *git checkout* to another branch. You can change the *HEAD* pointer manually by changing the *HEAD* file.

Figure 7 shows changes made to update the *HEAD* file to point to *branch2*.

If you now check the *git log*, you will find the head points to *branch2*.

How *git cherry-pick* works

git cherry-pick is a very popular and strong command. If you are not aware of this command, you can read about it from the official documentation of Git.

Just to give a brief explanation, if we have branch A and branch B and want to apply changes done in a certain commit of branch A to branch B, we need to just copy the commit object ID from branch A and do *git cherry-pick* over branch B.

I am currently on 'branch2', I will create two more commits. In the fourth commit, I will create a new sub-directory and add a file to it. In the fifth commit, I will just edit the *file2.txt* we created earlier.

Figure 8 shows the output of the *git log* of *branch2* after these changes.

Now I only want the changes made in the fifth commit to be copied to the *main* branch. So I will copy its commit ID, checkout to *main* branch and give the *git cherry-pick* command.

Figure 9 shows the output of the *git log* of the *main* branch before the command is given.